Practical-5 : Study and implement various operations on Python sets such as creation, insertion, deletion, traversal, set operations, copying, and comparison with other data structures.

Use a Python set ports to represent the network port numbers used in a computer system and perform the following tasks:

1. Populate the set ports with at least 10 network port numbers.

```
ports = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10,2000}
```

2. Display the complete set.

```
print(ports)
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 2000}
```

3. Demonstrate that duplicate port numbers are automatically removed when added to a set.

```
ports.add(1)
print(ports)
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 2000}
```

4. Find and display the total number of unique ports using len().

```
print(len(ports))
```

```
11
```

5. Check whether a specific port number exists in the set using the in operator.

```
print(1 in ports)
```

```
True
```

6. Add a new port number to the set using add().

```
ports.add(11)
print(ports)
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 2000}
```

7. Add multiple port numbers to the set using update().

```
ports.update([12, 13, 14, 15])
print(ports)
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000}
```

8. Remove a specific port number using remove().

```
ports.remove(1)
print(ports)
```

```
{2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000}
```

9. Remove a port number using discard() and observe the difference from remove().

```
ports.discard(1)
print(ports)

# Difference:
# remove() → gives error if element not found
# discard() → does NOT give error if element not found
```

```
{2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000}
```

10. Remove and return an arbitrary port number using pop().

```
popped_port = ports.pop()
print("Removed:", popped_port)
print("Remaining:", ports)
```

```
Removed: 2
Remaining: {3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000}
```

11. Traverse the set using a for loop.

```
for port in ports:
    print(port)
```

```
3
4
5
6
7
8
9
10
11
12
13
14
15
2000
```

```
2000
```

12. Create another set reserved_ports containing commonly reserved port numbers.

```
reserved_ports = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

13. Find and display the union of ports and reserved_ports.

```
print(ports.union(reserved_ports))
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000}
```

14. Find and display the intersection of ports and reserved_ports.

```
print(ports.intersection(reserved_ports))
```

```
{3, 4, 5, 6, 7, 8, 9, 10}
```

15. Find and display the difference between ports and reserved_ports.

```
print(ports.difference(reserved_ports))
```

```
{11, 12, 13, 14, 15, 2000}
```

16. Find and display the symmetric difference between ports and reserved_ports.

```
print(ports.symmetric_difference(reserved_ports))
```

```
{1, 2, 11, 12, 13, 14, 15, 2000}
```

17. Check whether reserved_ports is a subset of ports.

```
print(reserved_ports.issubset(ports))
```

```
False
```

18. Check whether ports is a superset of reserved_ports.

```
print(ports.issuperset(reserved_ports))
```

```
False
```

19. Create a copy of the set using the copy() method.

```
ports_copy = ports.copy()
print(ports_copy)
```

```
{3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000}
```

20. Demonstrate the difference between assignment and copying of sets.

```
#Copying
ports_copy2 = ports.copy()
ports_copy2.add(17)
print("Original:", ports)
print("Copied Set:", ports_copy2)

# Assignment
ports_copy1 = ports
ports_copy1.add(17)
print("Original:", ports)
print("Assigned Copy:", ports_copy1)
```

```
Original: {3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000}
Copied Set: {3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000, 17}
Original: {3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000, 17}
Assigned Copy: {3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000, 17}
```

21. Clear all elements from the set using clear().

```
ports.clear()
print(ports)
```

```
set()
```

22. Convert the set into a list and display the result.

```
ports_list = list(ports)
print(ports_list)
```

```
[]
```

23. Convert the set into a tuple and display the result.

```
ports_tuple = tuple(ports)
print(ports_tuple)
```

```
()
```

24. Create a new set containing only port numbers greater than 1024.

```
ports = {3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 2000}

ports_greater_than_1024 = {port for port in ports if port > 1024}
print(ports_greater_than_1024)
```

```
{2000}
```

25. Compare sets with lists and tuples with respect to uniqueness, ordering, and use cases.

```
numbers_set = {1, 2, 3, 3, 4}
numbers_list = [1, 2, 3, 3, 4]
```

```
numbers_tuple = (1, 2, 3, 3, 4)

print("Set:", numbers_set)
print("List:", numbers_list)
print("Tuple:", numbers_tuple)

print("Is set ordered?", "No")
print("Is list ordered?", "Yes")
print("Is tuple ordered?", "Yes")

print("Does set allow duplicates?", "No")
print("Does list allow duplicates?", "Yes")
print("Does tuple allow duplicates?", "Yes")
```

```
Set: {1, 2, 3, 4}
List: [1, 2, 3, 3, 4]
Tuple: (1, 2, 3, 3, 4)
Is set ordered? No
Is list ordered? Yes
Is tuple ordered? Yes
Does set allow duplicates? No
Does list allow duplicates? Yes
Does tuple allow duplicates? Yes
```